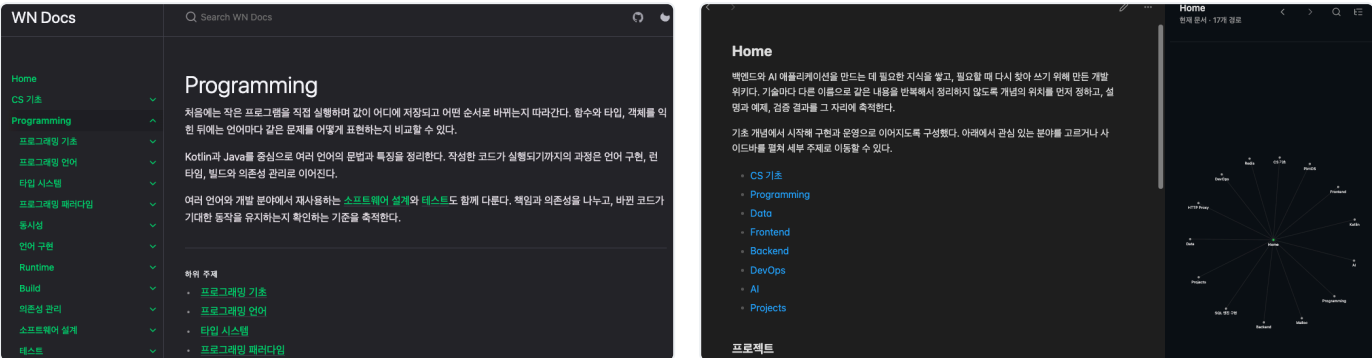


LLM 위키 시스템 (woon-core · WN Docs).

대화, 메모, 책을 출처 레코드와 주장 레코드로 나눠 저장하고, LLM 호출 없이 두 레코드를 결합해 위키 페이지를 생성하는 개인 지식 시스템입니다. 카탈로그 스키마, 컴파일러, 검색 색인, MCP 서버, Obsidian 플러그인 4종으로 구성됩니다.



승인된 문서만 투영해 공개하는 WN Docs 사이트입니다

Obsidian으로 여는 비공개 정보와 직접 구현한 링크 그래프입니다

항목	내용
기간	개인 프로젝트 · 1년 차 · 진행 중
역할	1인 · 설계 · 구현 · 검증
스택	Python · SQLite FTS5 · MCP · YAML
저장소	woonyong-choi/woon-core
검증	pytest 통과 · CI · gitleaks 0건

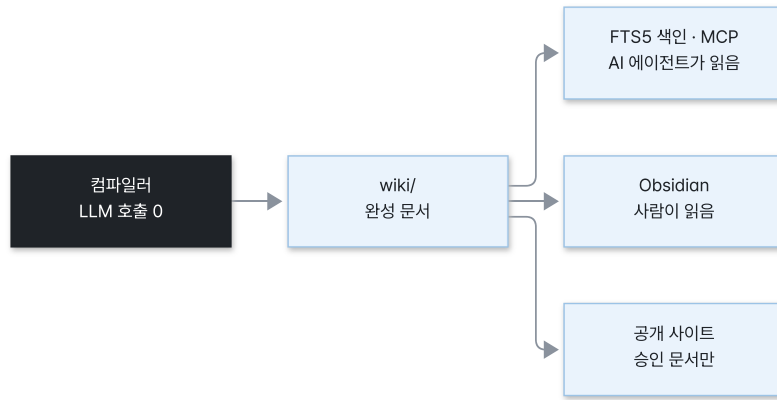
문제 정의

AI와 정리한 내용을 Obsidian에 쌓는 동안 같은 개념이 다른 이름으로 중복되고, 문장의 출처를 다시 찾기 어려워졌습니다. AI에게 문서 전체를 저작성하게 했을 때는 이전 내용이 사라지기도 했습니다. 검색 결과보다 문장의 근거를 먼저 확인할 수 있어야 했기 때문에, **검색보다 출처를 먼저 고정했습니다.**

시스템 구조: 원자료에서 공개 사이트까지

원자료는 레코드 3종을 거쳐 페이지가 됩니다. intake가 대화 로그와 메모, Docling으로 변환한 책, 음성 기록을 수집해 원본 위치와 해시를 보유한 `source_record` 를 생성합니다. 거기에서 추출한 한 덩어리가 `accepted_claim` 이 되고, `page_spec` 이 출력 경로 하나와 claim 순서만 정의합니다. 결정적 컴파일러는 그 순서대로 본문을 결합하고 입력·출력 해시를 receipt에 기록합니다.

컴파일 결과인 `wiki/` 폴더는 사람이 읽는 정보인 동시에 색인과 공개 투영의 입력입니다. 여기에서 세 갈래로 분기합니다. FTS5 색인과 MCP를 경유해 AI 에이전트가 읽고, 사람은 Obsidian에서 같은 폴더를 열며, 승인된 페이지만 공개 사이트로 배포됩니다. 색인은 필요한 시점에 생성되고 MCP 프로세스가 연결된 동안에만 유지되므로 상주 프로세스가 없습니다. 에이전트에 노출한 지식 도구는 13개이며, 읽기 도구는 전부 읽기 전용으로 선언했습니다.



컴파일된 wiki/ 폴더 하나가 검색 색인·Obsidian 열람·공개 사이트 세 갈래의 입력이 됩니다

데이터 모델: 레코드 다섯 종과 해시 연결

레코드는 5종입니다. 원자료 하나에 대응하는 `source_record`, 거기에서 추출한 `accepted_claim`, 출력 경로와 순서를 정의하는 `page_spec`, 컴파일 결과를 기록하는 `receipt`, 그리고 보관 본문의 승인을 남기는 `review-queue` 항목입니다. 레코드마다 해시를 보유하며 그 해시가 다음 레코드의 입력이 됩니다. `source`의 `normalized_sha256` 이 `claim`의 근거 목록에 참조되고, `claim`이 `page spec`의 순서 목록에 참조되며, 이 셋을 결합한 값이 `input_sha256`, 컴파일 결과 파일의 해시가 `output_sha256` 입니다. 한 단계라도 변경되면 이후 해시가 전부 달라지므로 재컴파일 전까지 검색이 해당 페이지를 반환하지 않습니다.

현재 카탈로그에는 `source` **12,914건**, `claim` **13,838건**, `page spec`과 `receipt`가 각 **2,422건** 등록되어 있습니다. 그중 보관 상태 `source` **7,684건**과 대체된 `claim` **7,673건**은 삭제하지 않고 보존합니다. 페이지의 특정 문장이 어느 레코드에서 파생되었는지 역추적하려면 이전 레코드가 필요하기 때문입니다. 데이터베이스 대신 YAML 파일을 선택한 이유도 `git log -p`로 변경 이력을 확인하기 위해서였고, 그 비용으로 `sources.yaml` 하나가 **100MB**를 초과합니다.



앞 레코드의 해시가 뒤 레코드의 입력이 되고, 마지막에 실제 파일과 바이트를 대조합니다

다섯 번째 레코드인 `review-queue` 항목은 보관 본문을 다시 컴파일에 투입할 때 사용합니다. 별도의 승인 화면은 구현하지 않고, 대화에서 진행을 승인하면 에이전트가 기록을 작성하도록 설계했습니다. 다만 그 기록이 충족해야 할 조건은 코드가 강제합니다.

```

src/woon_core/knowledge/compiled_wiki.py

# src/woon_core/knowledge/compiled_wiki.py
if (item.get("status") == "approved"
    and item.get("kind") == "manual-archive"
    and item.get("input_sha256") == body_hash
    and item["approved_by"].strip()):
    return
raise WoonError("compiled archive requires an approved review bound to the input hash")
  
```

결정적 컴파일러: LLM 없는 문서 생성

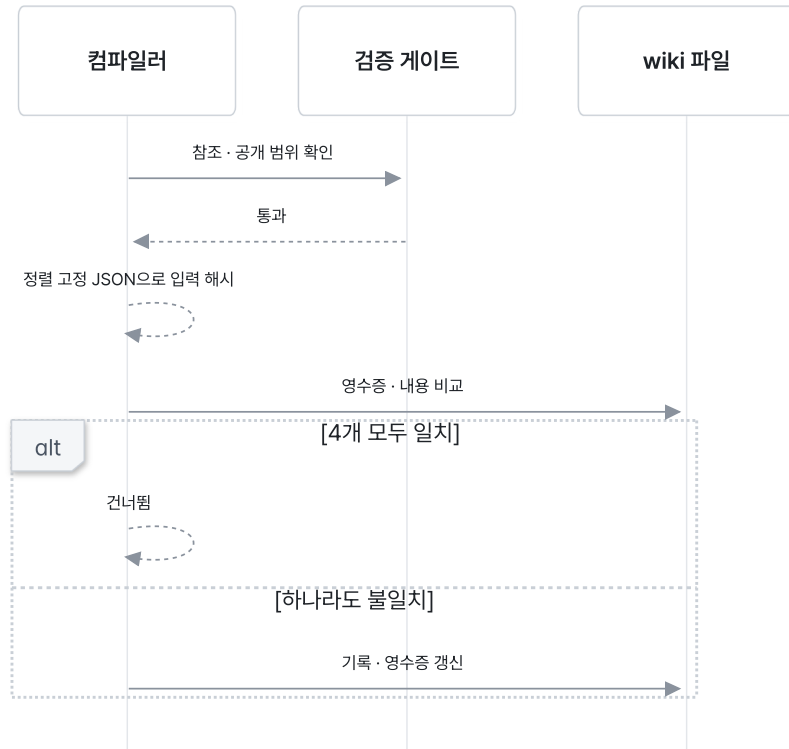
AI에게 페이지 전체를 재작성하게 하면 결과가 어느 원자료의 어느 문장에서 파생되었는지 알 수 없고, 두 번째 수정에서 첫 번째 결과가 소실됩니다. 그래서 편집 대상을 `wiki/` 외부 카탈로그에 두고 페이지는 언제나 복원 가능한 산출물로 구성했습니다. **AI가 본문을 작성하고 문단마다 출처 주석을 부여하는 방식**도 검토했지만, 동일 입력에서 동일 바이트가 생성된다고 보장할 수 없으면 출력 해시로 파일의 무결성을 판정할 수 없어 채택하지 않았습니다. 그 대가는 승인된 `claim`이 없는 주제는 페이지가 생성되지 않아 문서 증가 속도가 느리다는 점입니다.

결정성은 직렬화를 고정해야 확보됩니다. 입력 해시는 `json.dumps`에 `sort_keys=True`와 고정 구분자를 지정해 계산하며, 한 곳이라도 `dict` 순서에 의존하면 동일 입력에서 다른 해시가 산출됩니다. 컴파일 루프는 이 해시 하나만 검사하지 않습니다. 입력 해시, 컴파일러가 생성한 본문의 해시, 출력 파일의 해시, 디스크의 현재 파일 내용까지 **4개가 모두 일치할 때만 건너뛸**니다. 수동으로 편집한 출력은 마지막 조건에서 검출되어 다시 기록됩니다.

```
src/woon_core/knowledge/compiled_wiki.py
```

```
# src/woon_core/knowledge/compiled_wiki.py
if (not force and receipt is not None
    and receipt.get("input_sha256") == input_sha256
    and receipt.get("compiler_projection_sha256") == compiler_projection_sha256
    and receipt.get("output_sha256") == expected_hash
    and existing_text == output):
    unchanged += 1
    continue
```

4개를 한 번에 비교하므로 컴파일은 게이트 통과와 해시 계산, 파일 대조를 단일 경로로 처리합니다.



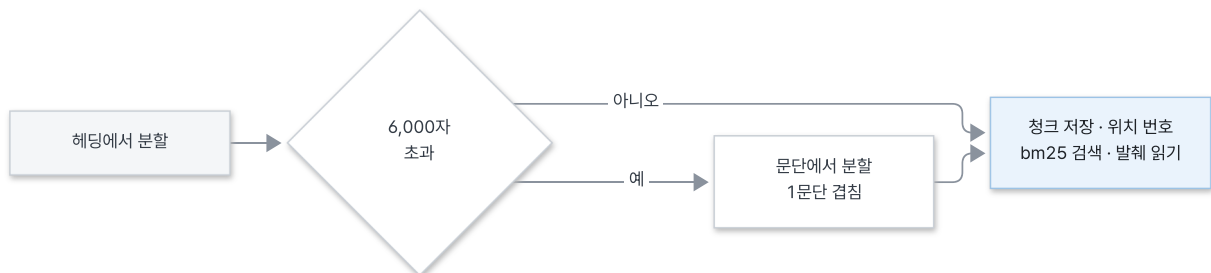
컴파일 1회가 게이트를 통과해 해시를 비교하고 기록하거나 건너뛰기까지의 순서입니다

게이트는 존재하는 source와 승인된 claim만 참조하도록 제한하고, 공개 페이지는 공개 source만 사용하게 하며, 보존 목적이 명시되지 않은 새 source를 거부합니다. 한 가지는 분명히 밝혀 두어야 합니다. 이 구조는 환각을 차단하지 않습니다. 컴파일러가 검증하는 대상은 claim이 참조하는 source의 존재 여부와 해시 일치이지, claim의 내용이 source에 실제로 존재하는지가 아닙니다.

틀린 문장을 차단하는 것이 아니라 어느 레코드에서 파생되었는지 역추적하게 하는 것이 목표였습니다.

검색 색인: 절 단위 청킹과 발췌 읽기

에이전트가 문서 전체를 문맥에 적재하지 않도록 절 단위로 분할합니다. 문서는 Markdown 헤딩에서 분할하고 그 경계에는 겹침을 두지 않습니다. 절 하나가 6,000자를 초과할 때만 문단 경계에서 재분할하며, 그 경우에만 다음 청크가 앞 청크의 마지막 문단으로 시작합니다. 절 경계는 의미의 경계이고 문단 경계는 길이 때문에 발생한 경계이므로 서로 다르게 처리해야 한다고 판단했습니다.



헤딩에서 분할하고, 6,000자를 초과하는 절만 문단 경계에서 한 문단을 겹치며 나눕니다

겹침 범위가 과도하게 확장되지 않도록 제한하는 조건은 함수 하나에 정의했습니다. 앞 문단이 청크 상한의 절반을 초과하거나, 이어 붙였을 때 다음 청크가 상한을 넘으면 겹침을 적용하지 않습니다.

```
src/woon_core/knowledge/adapters/sqlite_search.py
```

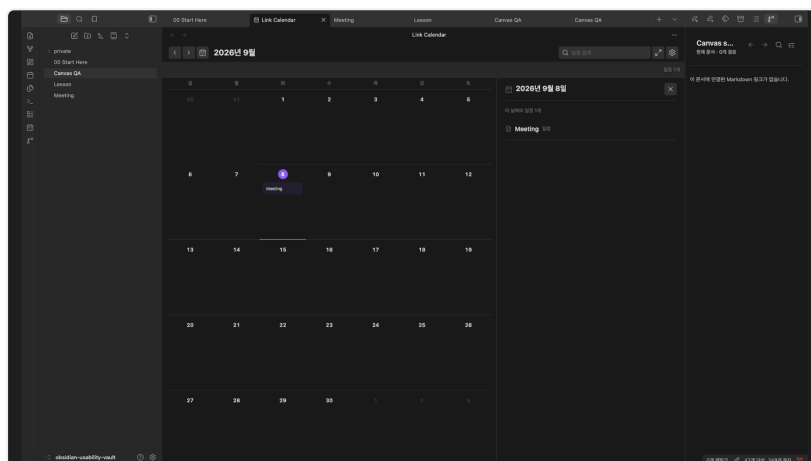
```
# src/woon_core/knowledge/adapters/sqlite_search.py
def _overlap(previous: str, paragraph: str, max_chars: int) -> tuple[str, ...]:
    """Carry the previous chunk's last paragraph forward when it stays small."""
    if (len(previous) > max_chars // 2
        or len(previous) + 2 + len(paragraph) > max_chars):
        return ()
    return (previous,)
```

효과는 직접 측정했습니다. [wiki/](#) 아래 Markdown **2,747**편을 색인한 뒤 질의 1건을 실행하고, 상위 결과가 지시한 문서 1편(본문 29,284자)을 읽는 데 소요되는 문자 수를 계측했습니다. 문서 전체를 읽으면 **29,873**자가 문맥에 적재되고, 검색 결과와 해당 절만 읽으면 **7,631**자로 **26%**, 앞뒤 이웃 절을 하나씩 추가하면 11,450자로 38%입니다. 문서 1편에 질의 1건, 1회 측정이므로 절이 짧은 문서에 서는 격차가 줄어듭니다.

검색을 벡터가 아니라 FTS5로 구현한 이유도 같은 맥락입니다. 색인이 단일 파일이고 재생성 비용이 낮으며 상주 프로세스가 불필요한데, 임베딩 모델을 구동하는 순간 연결된 동안에만 유지되는 도구라는 전제가 깨집니다. 초기 3일간은 Go로 벡터 어댑터까지 구현했다가 Python으로 전환하며 제거했고, 되돌린 이유는 커밋에 기록되어 있지 않아 여기에 단정해 서술하지 않겠습니다. 그 대가인 의미 검색 부재는 bm25 제목 가중치를 본문의 5배로 설정해 일부만 보완했습니다.

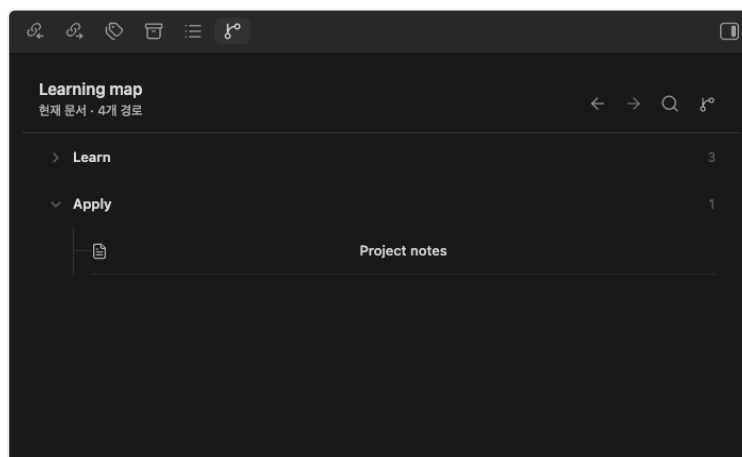
Obsidian 플러그인 네 종

별도 애플리케이션을 만들지 않고 기존 Vault의 [wiki/](#) 폴더를 컴파일러 출력으로 사용했습니다. 작성과 검색을 Obsidian 한곳에 유지하는 대신, 필요한 기능은 플러그인으로 보완했습니다. Code Blocks·Calendar·Graph는 Obsidian Community에 등록했고, Diagrams는 출시 후보를 검증하고 있습니다.



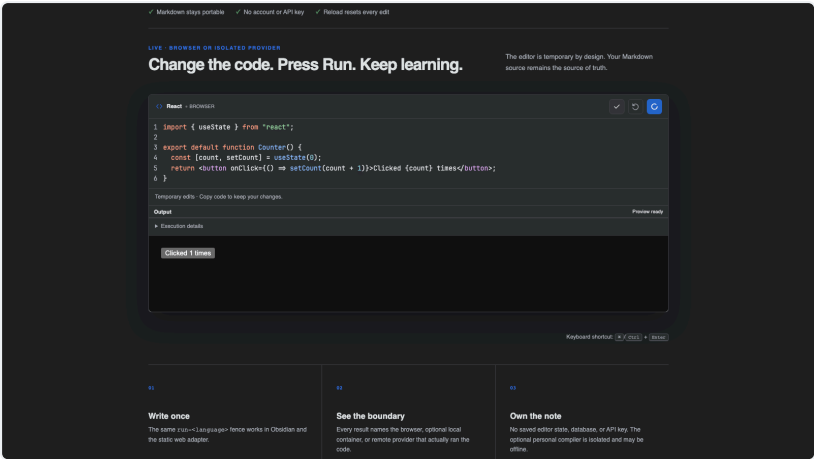
날짜가 기록된 노트를 달력에서 찾아 원본을 여는 화면입니다

Manta Calendar는 Vault의 날짜를 수집해 달력으로 표시하고 원본 노트를 엽니다. 위키는 개념 단위로 정리되지만 학습 기록은 시간 단위로 역추적해야 할 때가 있는데, Markdown에 기록한 날짜로 그날 정리한 내용을 찾을 방법이 없었습니다. 선택적인 Google Calendar 동기화까지 포함해 [woonyong-choi/manta-calendar](#)로 공개했습니다.



현재 문서가 작성한 링크를 순서대로 전개해 보여 주는 링크 그래프 패널입니다

Manta Graph는 현재 문서가 작성한 링크를 순서대로 따라가는 1단계 링크 그래프입니다. 기본 그래프는 전체 연결만 표시할 뿐, 작성 순서대로 확인하는 화면이 없었습니다. 부모와 자식 관계가 컴파일러가 생성한 구조이므로 그 순서대로 확인해야 문서 배치의 적절성을 판단할 수 있었습니다. 저장소는 [woonyong-choi/manta-graph](https://github.com/woonyong-choi/manta-graph)입니다.



코드 펜스를 샌드박스에서 실행해 출력까지 보여 주는 화면입니다

Manta Code Blocks는 문서의 코드 펜스를 샌드박스에서 실행하고 출력을 표시합니다. 편집 내용은 임시 상태로만 유지해 Markdown 원본을 바꾸지 않습니다. 저장소는 [woonyong-choi/manta-code-blocks](https://github.com/woonyong-choi/manta-code-blocks)입니다.

Manta Diagrams는 Mermaid 다이어그램을 Manta 레이아웃으로 렌더링하고 노트 안에서 확대·이동할 수 있게 합니다. Markdown 원본을 바꾸지 않고 다이어그램만 분리해 읽도록 구현했습니다. 저장소는 [woonyong-choi/manta-diagrams](https://github.com/woonyong-choi/manta-diagrams)입니다. 현재 v0.1.0 출시 후보를 검증 중이며 Community에는 등록하지 않았습니다.

검증 결과와 수치

카탈로그와 색인은 파일을 직접 계수했고, 토큰 수치는 실제 검색 어댑터로 1회 측정한 값입니다. 컴파일러는 페이지마다 통과한 검사 목록 (schema , source-provenance , accepted-claims , frontmatter-h1 , privacy)을 receipt에 함께 기록합니다. 그래서 이후에 어떤 게이트를 통과해 생성된 페이지인지 파일만으로 확인할 수 있습니다.

항목	값	조건
카탈로그	source 12,914건 · claim 13,838건 · page spec 2,422건	2026-09-28 YAML 직접 계수
공개 문서	240건 · 본문 완료 195건	공개 저장소 집계
검색 측정 스냅샷	문서 2,747편 · 절 9,374개 · 청크 9,409개	2026-09-22 측정 · 청크 상한 6,000자
토큰 경계	29,873자 → 7,631자 · 26%	문서 1편 · 절의 1건 · 1회
코드 규모	구현 78,185줄 · 테스트 50,042줄	공개 <code>woon-core</code> main · 2026-09-28 계수

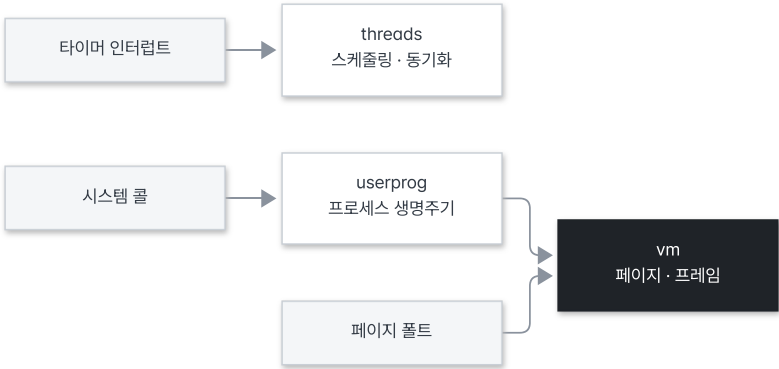
이전에 사용하던 공개 위키 **1,346건**은 사이트 빌드가 제외하던 키워드 등록 수였고, 현재의 **240건**은 실제로 렌더링되는 문서 수입니다. 컴파일 전체 소요 시간과 검색의 재현율·정밀도는 측정 기준을 확립하지 못해 계측하지 않았습니다.

범위와 한계

macOS에서 개인 작업 환경으로 사용하는 것을 전제로 구현했습니다. 그래서 다른 사용자가 자신의 Vault에 적용하는 설치 절차, 다중 사용자 승인, 동시 편집 처리처럼 범용 플랫폼이 갖춰야 할 설계는 아직 미흡합니다. 승인이 대화 1회와 기록 1건이고 동시 편집은 쓰기 전 revision 검사까지만 차단하는 현재 구조로는, 다수가 사용하는 환경에 그대로 적용할 수 없습니다.

x86-64 교육용 운영체제 커널 (Pintos).

스레드가 어떤 순서로 실행되고 프로세스가 메모리를 어떻게 분배받았다가 반환하는지를 커널 코드 수준에서 직접 검증한 프로젝트입니다. KAIST가 배포한 x86-64 Pintos 골격 위에 우선순위 스케줄링과 동기화, 프로세스 생명주기, 가상 메모리, Copy-on-Write fork를 구현했습니다.



커널 진입점 세 가지와 각 경로가 연결되는 디렉터리 구성입니다

항목	내용
기간	팀 과제 2026-04 ~ 05 · 개인 확장 08 ~ 09
역할	스케줄링 · 동기화 · 생명주기 · VM 분담
스택	C · x86-64 asm · QEMU · GitHub Actions
저장소	woonyong-choi/lrn-pintos
내 담당	스케줄링 · 우선순위 기부 · 생명주기 · SPT · 스왑 슬롯 · CoW 설계
팀원	파일 시스템 콜 · 프레임 테이블 · 교체 알고리즘 · mmap 본체
개인 확장	MLFQS · 고정소수점 연산 · x86 QEMU CI · 선점 버그 수정
규모	7명 · main 병합 기여자 6명 · 본인 커밋 159/214건(커널 소스 89건)

가상 주소 공간 설계

우선순위 역전이나 Copy-on-Write를 문헌으로 학습하는 것과, 해당 개념이 무너지는 조건을 직접 재현하는 것은 다릅니다. 그래서 스케줄러가 판단을 그르치는 지점, 부모 프로세스가 자식보다 먼저 종료되는 지점, 물리 프레임이 부족해 기존 페이지를 교체해야 하는 지점을 코드로 구현했습니다. 이 재현의 출발점이 가상 주소 공간입니다.

유저 영역과 커널 영역을 구분하는 경계는 `KERN_BASE` (0x8004000000) 하나입니다. 커널은 그 상단에 물리 메모리 전체를 선형으로 매핑하므로, 커널 가상 주소와 물리 주소가 단순 가감산만으로 변환됩니다. 경계 아래가 유저 영역이며, 스택은 `USER_STACK` (0x47480000)에서 하위 방향으로 확장되고 코드 세그먼트는 0x400000에 고정되며, 그 사이 구간은 대부분 매핑되지 않은 영역입니다.

스택 포인터 하위의 매핑되지 않은 주소에 접근했다고 해서 모두 스택 확장 대상인 것은 아닙니다. 잘못된 포인터 접근을 스택 확장으로 오인하면 강제 종료해야 할 프로세스에 메모리를 할당하게 됩니다. 따라서 유저 스택 범위 내부인지, 1MB 상한을 초과하지 않는지, 현재 `rsp` 기준 32바이트 이내인지 세 조건을 모두 검증한 뒤에만 새 페이지를 할당합니다. 32바이트의 여유 오프셋은 하드웨어의 동작 특성을 수용하기 위한 것으로, `PUSH` 계열 명령이 `rsp`를 감소시키기 전에 접근 검사를 수행하므로 정상적인 스택 연산에서도 fault 주소가 `rsp` 하단에 기록될 수 있기 때문입니다.

pintos/vm/vm.c

```
/* pintos/vm/vm.c */
static bool vm_should_grow_stack(struct intr_frame *f, void *addr, bool user) {
    uintptr_t fault_addr = (uintptr_t) addr;
    uintptr_t rsp = user ? (uintptr_t) f->rsp : thread_current()->user_rsp;
    return fault_addr < (uintptr_t) USER_STACK &&
        fault_addr >= (uintptr_t) (USER_STACK - STACK_MAX) &&
        fault_addr >= rsp - 32;
}
```

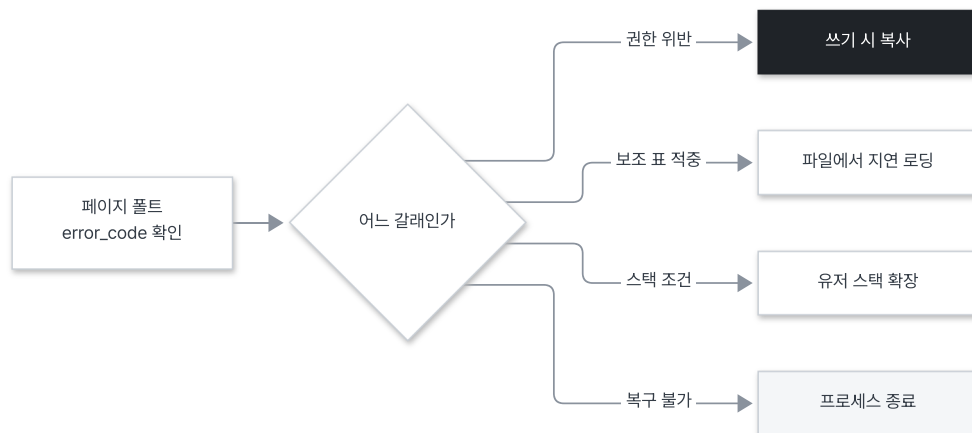
스레드 구조체와 page fault 처리 경로

스레드 구조체의 총 24개 필드 중 5개는 우선순위 기부, 3개는 고급 스케줄러(MLFQS) 구현을 위해 직접 추가했습니다. 아래는 그 가운데 기부와 스케줄러에 관여하는 필드만 발췌한 것입니다. 설계를 결정한 요인은 개념이 아니라 하드웨어의 제약이었습니다. CPU는 fault가 발생한 주소를 CR2 레지스터에만 기록하고 원인은 `error_code`의 세 비트로만 전달하므로, 해당 비트를 해석해 분기를 결정하는 것이 커널의 첫 작업이 됩니다. 매핑 자체가 없는 fault와 매핑은 존재하나 권한이 일치하지 않는 fault를 구분하는 이 비트가 그대로 Copy-on-Write의 진입점이 되었습니다.

pintos/include/threads/thread.h

```
/* pintos/include/threads/thread.h */
struct thread {
    int priority;                // 현재 유효 우선순위
    int base_priority;           // 우선순위 기부 전 원래 우선순위
    struct list donation_list;   // 나에게 우선순위를 기부한 스레드 목록
    struct lock *waiting_lock;   // 현재 기다리는 락, 우선순위 기부 전파용
    int recent_cpu;              // MLFQS recent_cpu (17.14 fixed-point)
};
```

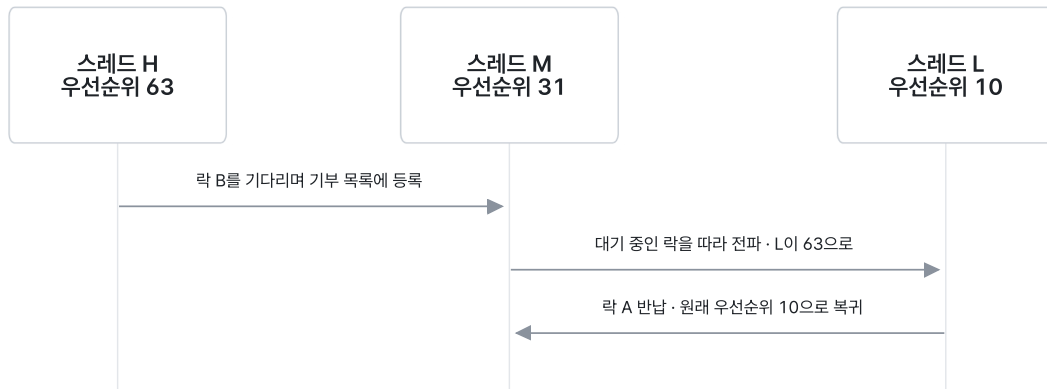
분기 조건을 이 순서로 배치한 이유는 처리 비용을 최적화하기 위해서입니다. 복구가 불가능한 주소를 먼저 차단하고, 새 페이지 할당이 필요한 스택 확장을 마지막 단계에 배치했습니다. 보조 페이지 테이블을 해시로 구성한 것도 같은 맥락으로, 이 조회는 fault마다 한 번 발생하며 필요한 연산이 단건 탐색뿐이어서 범위 조회가 요구되지 않습니다.



`error_code`와 보조 페이지 테이블 조회 결과가 fault를 네 갈래로 분기시킵니다

우선순위 기부와 반납

낮은 우선순위 스레드 L이 락을 보유한 상태에서 중간 우선순위 M에 밀릴 경우, 해당 락을 대기하는 높은 우선순위 H가 M보다 늦게 완료되는 우선순위 역전(Priority Inversion) 현상이 발생합니다. 해결책은 H의 우선순위를 락 보유자에게 일시적으로 위임하는 것입니다. 다만 락이 중첩되면 대기 중인 락을 따라 한 단계 더 전파해야 하며, 순환 대기에서 커널이 정지하지 않도록 최대 7단계까지만 전파하도록 제한했습니다. 이는 순환을 탐지하는 대신 전파 깊이를 제한한 선택이므로 완전한 해법은 아닙니다.



우선순위 기부가 락을 건너 전파되고 반납 시 해당 기부분만 제거됩니다

구현 난이도가 높았던 지점은 우선순위의 전파가 아니라 반납(Restore) 처리였습니다. 기부받은 값을 누적 저장하면 락이 중첩될수록 복원 연산이 복잡해지므로, 누적하지 않고 락을 반납할 때마다 `base_priority` 에서 다시 시작해 잔여 기부의 최댓값만 반영하도록 구현했습니다.

pintos/threads/thread.c

```

/* pintos/threads/thread.c */
void refresh_priority (struct thread *t) {
    t->priority = t->base_priority;
    if (!list_empty (&t->donation_list)) {
        list_sort (&t->donation_list, thread_priority_donation_elem, NULL);
        /* ... 맨 앞(최댓값)이 더 크면 그 값으로 올린다 */
    }
}

```

이 함수가 목록에 등록된 스레드의 우선순위를 호출 시점마다 조회하므로, 중첩 기부의 복원이 단일 함수 안에서 완결됩니다. 트레이드오프는 락 반납마다 목록 정렬이 발생한다는 점입니다. 같은 이유로 세마포어와 조건 변수도 대기자 등록 시점의 값을 캐시하지 않고 깨우는 시점에 실제 값을 조회하도록 변경했습니다. 고급 스케줄러는 다른 제약을 받았는데, 커널에 부동소수점 연산이 없어 정수 나눗셈만 사용하면 59/60이 0으로 계산되어 감쇠 계수의 유효 자릿수가 소실됩니다.

32비트 정수를 정수부 17비트와 소수부 14비트로 분할해 사용한 이유입니다. 소수부 14비트는 1/16384 단위까지 표현하므로 해당 계수의 유효 자릿수를 보존합니다.

Copy-on-Write fork

기본 `fork` 는 부모의 모든 페이지를 자식에게 복사하지만, `fork` 직후 `exec` 을 호출하는 일반적인 패턴에서는 해당 복사가 전부 폐기됩니다. 과제 요구 범위 밖이었으나 이 비효율을 제거하기로 했습니다. `fork` 직후에는 부모와 자식의 페이지 테이블이 동일 프레임을 읽기 전용으로 참조하며 참조 수가 2가 됩니다. 이후 한쪽이 쓰기를 시도하면 쓰기 보호 fault가 발생하고, 이때 참조 수만으로 복사 여부를 결정합니다. 참조 수가 2 이상이면 새 프레임을 확보해 4kB를 복사한 뒤 기존 프레임의 참조 수를 감소시키고, 1이면 복사 없이 쓰기 권한만 복원합니다.

pintos/vm/vm.c

```

/* pintos/vm/vm.c */
static bool
vm_handle_cow (struct page *page) {
    struct frame *old_frame = page->frame;
    if (old_frame->ref_count > 1)
        return vm_copy_cow_page (page, old_frame);
    return vm_remap_page (page, old_frame, true);
}

```

`memcpy` 는 기존 프레임을 읽기만 하며, 해당 프레임은 참조 수가 2 이상이므로 교체 대상에서도 제외됩니다. 따라서 4kB를 복사하는 동안 프레임 락을 점유할 필요가 없었고, 락 구간은 실제로 경합이 발생하는 참조 수 감소 한 줄로 축소했습니다. 설계와 최초 구현은 제가 담당했으며, 동일 파일을 함께 수정하던 팀원의 브랜치를 기준으로 삼아 제 처리 로직을 재적용한 뒤 통합 커밋을 작성했습니다.

테스트를 통과한 채 남아 있던 결함 세 건

세 건 모두 제가 유입시킨 결함이며, 과제 테스트를 전부 통과하는 동안 그대로 잔존했습니다. 발견 계기 역시 테스트가 아니라 후속 기능 구현이었습니다. 가장 오래 잠복한 것은 선점 버그입니다. 고급 스케줄러를 활성화한 시점에 기존에 통과하던 테스트가 실패했는데, 실패

항목이 모두 생성 직후 실행 순서를 검증하는 테스트여서 스레드 생성 경로를 분석했습니다. 새 스레드의 우선순위가 현재 스레드보다 높을 때 양보 여부를 판단하는 로직이, 실제로 설정된 값이 아니라 호출자가 전달한 인자를 참조하고 있었습니다.

pintos/threads/thread.c

```
/* pintos/threads/thread.c */
thread_unblock (t);

/* priority(인자)가 아니라 t->priority(실제 값)를 본다 */
if (t->priority > thread_current ()->priority) {
    thread_yield ();
}
return tid;
```

두 값이 분기되는 조건은 한 가지뿐입니다. 고급 스케줄러가 활성화된 경우 생성 직후 우선순위를 재계산해 필드에 반영하므로, 그 시점에 만 인자와 실제 필드의 값이 달라집니다.



인자와 실제 필드는 기본 스케줄러에서 일치하며 MLFQS에서만 분기됩니다

직접 원인은 동일한 명칭의 인자와 필드를 혼동한 것이고, 근본 원인은 기본 스케줄러에서 두 값이 항상 일치해 테스트가 구조적으로 차이를 노출하지 못한 것입니다. 비교 대상을 실제 필드로 변경한 뒤, 검증 상태를 전달받은 기록이 아니라 직접 실행한 결과로 대체하기 위해 두 스위트를 별도 job으로 실행하는 CI를 구축했습니다. 나머지 두 건도 동일한 양상이었습니다. 우선순위 기부를 적용하자 `priority-donate-sema` 와 `priority-condvar` 가 실패했는데, 기부 로직 자체는 정상이고 깨우는 순서만 어긋나 대기 목록을 의심하기까지 시간이 소요되었습니다. 스왑을 적용한 뒤에는 `swap-anon` 계열을 반복 실행할수록 가용 슬롯이 감소하는 현상을 확인했습니다.

- **선점 버그** — 인자와 필드의 혼동. 두 값이 분기되는 스케줄러가 없어 미노출
- **대기자 우선순위 캐시** — 동일 값의 이중 저장과 부분 갱신. 조회 시점 원본 참조로 수정
- **스왑 슬롯 누수** — 슬롯 수명을 재적재 경로에만 결합. 해제 경로에서도 반납하도록 수정

인자명과 필드명에 동일한 명칭을 사용한 것은 부주의였으며, 단일 테스트 스위트가 모든 스케줄링 조합을 검증해 주리라 신뢰했던 점을 되짚는 계기가 되었습니다.

검증 결과와 수치

CI의 세 job이 QEMU x86 환경에서 threads-userprog-vm 검사를 나눠 실행합니다. 한쪽이 실패해도 다른 결과를 독립적으로 확인하기 위해 job을 분리했습니다. 동일 트리를 로컬 Docker에서 재실행해 같은 수치를 확인했으며, 선점 버그를 겪은 이후로는 전달받은 기록이 아니라 직접 실행한 값만 기재합니다.

스위트	테스트 수	조건
threads	28/28	<code>make -C pintos/threads check</code>
user-programs	95/95	<code>make -C pintos/userprog check</code>
virtual-memory	142/142	<code>make -C pintos/vm check</code> · threads 검사를 포함해 수치 중복

검증 범위가 QEMU 단일 환경의 과제 스위트에 한정되므로, 선점 버그처럼 스위트가 구조적으로 포괄하지 못하는 조합은 통과 여부와 무관하게 잔존합니다.

범위와 한계

파일 시스템 과제는 구현하지 않아 버퍼 캐시와 확장 가능한 파일, 하위 디렉터리가 모두 제외되어 있습니다. 장치 드라이버와 보안 모델, 다중 코어도 다루지 않았으며, 단일 CPU에서 인터럽트를 일시 비활성화하는 방식으로 보호한 구간은 다중 CPU 환경에 그대로 적용할 수 없습니다. 고아 프로세스 회수 실험에 사용한 테스트 5건은 후속 과제와 충돌하지 않도록 삭제했으나, 삭제 대신 명칭을 변경해 보존했어야

했습니다. 공유 중인 프레임의 스왑 아웃도 범위 밖으로 남겼습니다. 참조 수는 참조 개수만 제공할 뿐 참조 주체를 식별하지 못하는데, 프레임을 반출하려면 해당 프레임을 참조하는 모든 페이지 테이블에서 매핑을 해제해야 하기 때문입니다.

값의 출처를 헤더와 하드웨어 동작까지 추적해 확인하는 습관과 테스트 통과를 정확성의 근거로 삼지 않는 태도, 이 두 가지가 남았습니다.

디스크 기반 SQL 엔진 (lrn-sql).

SQL 문장을 파싱하고 실행 계획을 수립한 뒤 디스크의 레코드를 읽고 쓰는 단일 테이블 데이터베이스입니다. B+Tree 인덱스, 슬롯 힙 페이지, 256 프레임 버퍼 풀, 행·범위 잠금을 C11로 구현했습니다.

항목	내용
기간	팀 과제 3주 · 개인 정리 6주
역할	5인 팀 · 엔진과 서버 C 코드 전부
스택	C11 · POSIX pthread · GNU Make · GitHub Actions
저장소	woonyong-choi/lrn-sql · 옛 이름 MiniDB
검증	<code>make test-all</code> 510/510 · CI ASAN·UBSAN
내 담당	파서 · 플래너 · 실행기 · B+Tree · 힙 · pager · 잠금 테이블 · HTTP 서버
팀원	학습 문서 · 질문 정리 · C 코드 커밋 없음
개인 확장	범위 스캔 · 100만 행 벤치마크 · $O(N^2)$ 결합 2건 · CI 게이트

계층 구조와 잠금 순서

엔진을 5개 계층으로 분리하면서 각 계층에서 정의한 것은 기능이 아니라 책임 경계였습니다. 인덱스는 레코드의 내용을 알 필요 없이 위치만, 힙은 인덱스 키를 알 필요 없이 슬롯만, pager는 페이지의 종류를 알 필요 없이 바이트만 다루면 됩니다. 잠금도 동일한 원칙으로 4개 계층으로 분리했습니다. 행 잠금은 `id` 값이 문장 실행 중에 변경되지 않는다는 논리적 보장이고, 페이지 래치는 해당 페이지를 다른 스레드가 부분적으로 갱신한 상태로 노출하지 않는다는 물리적 보장이라 보호 대상이 다릅니다. 두 계층을 하나로 통합하면 레코드 하나를 읽는 동안 페이지 전체가 잠기게 됩니다.



문장 하나가 parser-planner-executor를 거쳐 bptree-table-pager로 전달되는 경로입니다

문장의 경계를 정의하는 지점은 `db.c` 한 곳이며, 잠금을 획득하고 해제하는 순서가 이 함수에 그대로 드러납니다. 엔진 RW 잠금을 해제한 뒤 마지막으로 보유 잠금을 일괄 해제하는데, 이 한 줄이 뒤에서 다룰 두 번째 결합의 기준이 됩니다.

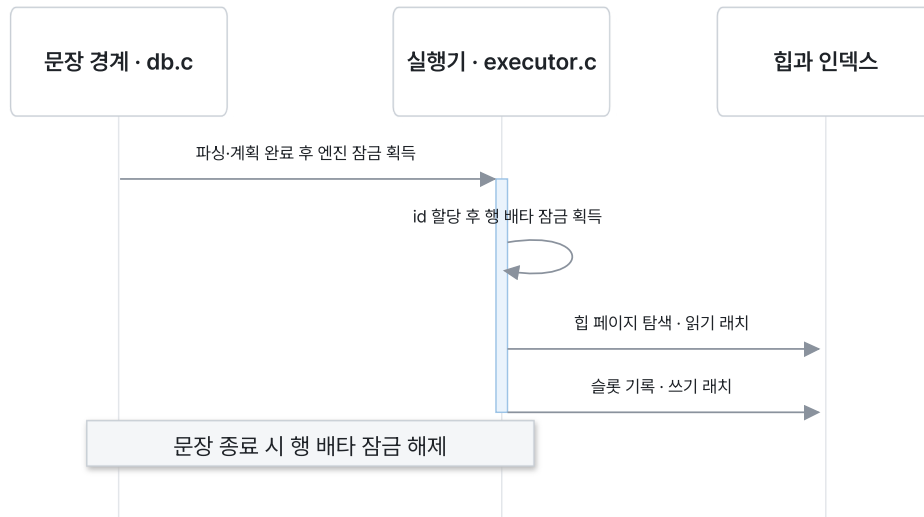
src/db.c

```
/* src/db.c */
if (is_ddl(stmt.type)) pthread_rwlock_wrlock(&engine_lock);
else pthread_rwlock_rdlock(&engine_lock);

res = execute(pager, &stmt);

pthread_rwlock_unlock(&engine_lock);
lock_release_all(&g_lock_table);
```

진입 경로는 두 가지로, 서버는 `db_execute()` 를 호출하고 REPL은 `execute()` 를 직접 호출합니다. 이 차이가 뒤에서 다룰 두 번째 결합의 원인입니다. INSERT 문장 하나를 추적하면 잠금이 상위 계층일수록 길게, 하위 계층일수록 짧게 유지되는 것을 확인할 수 있습니다. 엔진 RW 잠금은 실행 전 구간에서 유지되고, 행 배타 잠금은 실행기가 `id` 를 할당한 직후 획득해 문장 종료 시점에 해제됩니다. 페이지 래치는 단일 단계 안에서 획득과 해제가 끝나므로, 힙 래치와 인덱스 래치를 동시에 보유하는 구간이 없어 순서 역전에 의한 교착이 구조적으로 발생하지 않습니다.



행 배타 잠금은 id 할당부터 문장 종료까지, 페이지 래치는 단일 단계에서만 유지됩니다

B+Tree 인덱스와 힙 페이지 설계

인덱스 자료구조로 Hash 대신 B+Tree를 선택했습니다. BETWEEN 이나 `id > 5000` 같은 범위 질의를 지원하려면 키가 정렬된 상태로 저장되어야 하는데 Hash는 순서를 보존하지 못하며, 노드 하나를 디스크 페이지 하나에 대응시키기에도 B+Tree가 적합합니다. 대가는 점조회 성능이 Hash보다 낮고 분할·병합·언더플로 복구를 전부 직접 구현해야 한다는 것입니다. 리프 노드만 실제 레코드 위치를 보유하고 내부 노드는 라우팅만 담당하며, 리프끼리 연결되어 있어 한 번 진입하면 트리를 재탐색하지 않고 순차 순회할 수 있습니다.

리프 노드가 보유하는 값은 메모리 포인터가 아니라 파일 내 좌표입니다. 포인터는 프로세스를 재시작하면 무효가 되지만, 14바이트 리프 엔트리 가운데 6바이트를 차지하는 이 좌표는 파일을 다시 열어도 동일한 레코드를 가리킵니다.

```

include/storage/page_format.h

/* include/storage/page_format.h */
typedef struct {
    uint32_t page_id; /* 행이 저장된 힙 페이지 ID */
    uint16_t slot_id; /* 페이지 내 슬롯 번호 */
} __attribute__((packed)) row_ref_t;
  
```

힙 페이지에서는 8바이트 슬롯 디렉터리가 선두부터, 레코드 본문이 말미부터 서로 마주 보며 증가하고 양쪽이 만나면 해당 페이지는 포화 상태가 됩니다. 슬롯이라는 간접 계층을 둔 이유는 외부에는 슬롯 번호만 노출하고 그 슬롯이 가리키는 오프셋은 페이지 내부에서 자유롭게 변경할 수 있도록 하기 위해서입니다. DELETE는 레코드를 삭제하지 않고 슬롯 상태만 변경해 빈 슬롯 체인에 연결합니다. 다만 어느 페이지에 빈 슬롯이 있는지는 페이지 내부에만 기록되고 외부에는 존재하지 않는데, 이 설계가 뒤에서 다룰 첫 번째 결함의 원인이 됩니다.

버퍼 풀 교체와 동시성 제어

버퍼 풀은 256개 프레임으로 구성되며 교체 정책은 LRU입니다. 어떤 프레임을 축출할지는 `pin_count`와 `used_tick`이 결정하는데, 설계 과정에서 가장 혼동하기 쉬웠던 지점은 pin과 latch의 구분이었습니다. pin은 해당 프레임을 교체 대상에서 제외하라는 의미이고 latch는 현재 페이지 내용을 읽거나 쓰는 중이라는 의미라 보호 대상이 다릅니다. 실제로 두 개념을 혼용한 테스트 코드가 unpin만 반복 호출해 읽기 래치가 해제되지 않는 교착을 유발한 사례가 있었습니다.



교체 대상은 사용 중이 아닌 프레임 가운데 최종 접근이 가장 오래된 것입니다

서버 모델과 잠금 정책은 하루 사이에 세 차례 반복되었습니다. 고정 스레드 풀로 시작했다가 소켓 고갈을 해결하기 위해 keep-alive를 도입했으나, keep-alive 연결 16개가 워커 4개를 점유해 나머지 요청이 처리되지 못하는 head-of-line blocking이 발생했습니다. 결국 연결당 스레드 모델로 전환하면서 같은 날 오전에 작성한 스레드 풀 코드를 당일 제거했고, 최대 연결 128개에 스택 상한 약 1GB라는 비용을 수용했습니다. 남은 문제는 writer starvation이었습니다. 보유 중인 잠금만 충돌 검사에 반영하면 대기 중인 쓰기 요청이 있어도 신

규 읽기 요청이 계속 선점하기 때문입니다. 다음 날 대기 writer 수를 집계하는 카운터와 조건 하나로 해결했으며, 3초를 초과하면 교착으로 판단해 타임아웃을 반환합니다.

```
src/server/lock_table.c
```

```
/* src/server/lock_table.c */
static int reader_should_wait(lock_table_t *lt, lock_mode_t mode)
{
    return mode == LOCK_S && lt->waiting_writers > 0;
}
```

성능 결함 진단: 100만 행 삽입

데이터 측정 규모를 키우자 INSERT 시간이 5만 건 0.131초, 10만 건 0.461초, 20만 건 1.901초, 40만 건 8.415초로 증가했습니다. 레코드 수가 2배가 될 때 시간이 약 4배가 되는 $O(N^2)$ 에 가까운 증가였습니다. 코드 검토만으로는 원인을 특정하지 못해 실행 중인 삽입 프로세스에 gdb를 연결해 스택을 4회 샘플링했는데, 4회 모두 잠금 테이블의 체인 탐색 구간이었습니다. 단일 연결 벤치마크에서 잠금 경합이 병목일 수 없다는 점에서, 이때 잠금이 누적되고 있음을 확인했습니다.

첫 번째 결함은 힙 체인 재탐색입니다. 삽입 대상 페이지를 탐색하는 함수가 말미 페이지 포화 시 빈 슬롯을 재활용하기 위해 체인 전체를 처음부터 재순회하는데, DELETE가 한 번도 발생하지 않은 순차 INSERT에서도 동일하게 동작했습니다. 1회 호출 비용만 보고 호출 빈도를 고려하지 않은 결과였습니다. 보수적 힌트를 추가하고, 힌트의 전제가 바뀌는 4개 지점을 함께 갱신했습니다.

- **설정** — DELETE가 빈 슬롯을 생성하는 시점
- **해제** — 체인 전체를 순회했으나 빈 슬롯이 없는 경우
- **복원** — DB 재개방 시 체인을 순회해 실측값으로 복구
- **초기화** — DROP TABLE 로 힙 체인 자체가 교체되는 경우

```
src/storage/table.c
```

```
/* src/storage/table.c */
#ifdef MINIDB_DISABLE_FREE_HINT
    if (!pager->heap_may_have_free_slots) {
        return 0; /* 삭제된 적 없음 → 재활용할 슬롯도 없음 */
    }
#endif
```

두 번째 결함은 REPL의 잠금 미해제입니다. 서버 경로는 문장 종료 시 잠금을 일괄 해제하지만 REPL은 실행 함수를 직접 호출하면서 해제 단계를 누락했고, 벤치마크는 이 REPL 경로로 실행되었습니다. 잠금 테이블은 256개 해시 버킷으로 구성되므로 100만 개의 엔트리가 누적되면 버킷당 체인 길이가 약 3,900개에 이르고, 매 INSERT마다 이 체인을 탐색하므로 전체 복잡도가 $O(N^2)$ 이 됩니다. 커밋 이력을 확인해 보니 REPL은 행 잠금 개념이 도입되기 이전에 작성되었고, Strict 2PL을 경계 함수에 통합한 작업은 db.h · db.c · server.c 3개 파일만 수정했습니다. REPL이 규약을 위반한 것이 아니라 규약 도입 시점에 REPL을 함께 갱신하지 않은 것이며, 성능 문제로 나타났을 뿐 실제로는 Strict 2PL이 깨진 정확성 결함이었습니다. 수정은 문장 실행 직후 lock_release_all() 한 줄을 추가하는 것이었습니다.

두 결함이 당시 224건의 테스트를 통과한 이유는 기존 검사가 해당 경로를 밟지 않았기 때문입니다. 문장을 종료까지 실행하는 검사는 경계 함수를 경유해 잠금을 빠짐없이 해제했고, REPL처럼 실행 함수를 직접 호출하는 스위트는 잠금 테이블을 검사하지 않았습니다. 슬롯 재활용 테스트도 44바이트 레코드 100건만 넣어 힙 페이지 2장에 그쳤으므로 재탐색 비용이 드러나지 않았습니다. 이후 REPL 잠금 해제와 순차 INSERT 재탐색을 각각 깨뜨리면 실패하는 회귀 테스트를 추가했습니다.

함수 자체는 검증했지만, 그 함수가 호출되는지는 검증하지 않았습니다.

검증 결과와 수치

성능 수치는 단일 연결·합성 데이터·캐시 상주·3회 중앙값으로 측정했습니다. 단일 규모의 배수는 실행 환경에 따라 크게 흔들려 최신 문서에서는 여러 입력 크기의 log-log 기울기로 알고리즘 변화를 비교합니다. INSERT 개선 전 값은 수정 커밋 직전 소스를 같은 컴파일러로 다시 빌드했고, 범위 조치는 현재 소스에서 인덱스 경로만 끄는 컴파일 가드를 사용했습니다.

측정	개선 전 기울기	개선 후 기울기	조건
INSERT	$O(N^2.00)$	$O(N^1.19)$	5만~40만 행 · <code>make bench</code>
범위 조회	$O(N^0.88)$	$O(N^0.19)$	각 1,000행 반환 · 힙 스캔 대 B+Tree 리프 순회
현재 단일 규모	—	INSERT 632,962 · Range 3,267 ops/sec	100만 행 · 2026-09-22 · Apple M4 · median of 3
회귀 검사	—	510/510 passed	ASAN·UBSAN CI 포함

```
src/sql/planner.c

/* src/sql/planner.c */
static inline bool index_range_enabled(void)
{
#ifdef MINIDB_DISABLE_INDEX_RANGE
    return false;    /* 개선 전 경로를 같은 소스로 재현 */
#elseif
    return true;
}
}
```

단일 규모의 절대 수치는 캐시와 메모리 대역폭에 따라 크게 달라지므로 성과의 중심에 두지 않습니다. 대조군 PostgreSQL 16은 `fsync=off` 로도 WAL과 다중 버전, 복구 비용을 부담하지만 이 엔진에는 WAL과 crash recovery가 없습니다. 같은 INSERT라는 이름만으로 성능 우위를 주장할 수 없으며, 대조 실험은 구조적 병목을 찾기 위한 기준선으로만 사용했습니다.

범위와 한계

운영 데이터베이스의 요건 대부분은 구현 범위 밖입니다. WAL과 crash recovery가 없어 정상 종료 후 재개방은 보장하지만 예기치 않은 중단 이후의 복구는 다루지 않았고, MVCC와 격리 수준도 없어 읽기와 쓰기가 서로를 차단합니다. 다중 문장 트랜잭션 대신 문장 단위 autocommit만 지원하며, secondary index가 없어 `id` 외의 컬럼 조건은 플래너가 전체 스캔으로 라우팅합니다. 엔진 RW 잠금이 모든 쓰기를 직렬화해 페이지 단위 병렬성을 활용하지 못하는 제약도 남아 있습니다.

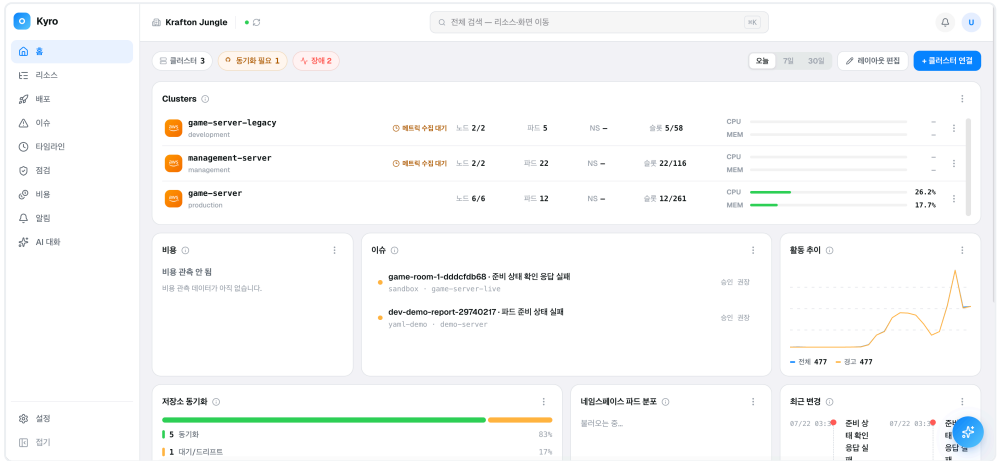
```
src/sql/planner.c

/* src/sql/planner.c */
case STMT_DELETE:
    if (stmt->predicate_kind == PREDICATE_ID_EQ)
        plan.access_path = ACCESS_PATH_INDEX_DELETE;
    else
        plan.access_path = ACCESS_PATH_TABLE_SCAN;
    break;
}
```

측정에도 한계가 있습니다. 100만 건 기준으로 DB 파일이 52MB이므로 전량 캐시에 적재되어, 디스크가 병목이 되는 구간은 아직 측정하지 못했습니다. 현재 회귀 테스트는 REPL 문장 종료 뒤 잠금이 남는 경우와 DELETE 없는 순차 INSERT가 힙 체인을 재탐색하는 경우를 막지만, 더 큰 데이터가 페이지 캐시를 벗어나는 조건은 검증하지 않습니다.

Kubernetes 장애 진단·복구 제안 서비스 (Clue).

Clue는 Kubernetes 장애 증거를 읽기 전용으로 수집하고, 버전이 부여된 규칙으로 원인을 판정합니다. 수정안은 GitHub Draft PR로만 제안하며, 배포 뒤 회복 여부를 확인하도록 설계했습니다. 5인 팀의 팀장으로 전체 아키텍처와 이벤트 런타임, 서비스 간 계약, CI와 배포를 담당했습니다. 공개 정리본은 이 흐름을 계약 테스트로 검증했으며, 외부 클러스터와 GitHub App을 연결한 종단 E2E는 아직 검증하지 않았습니다.

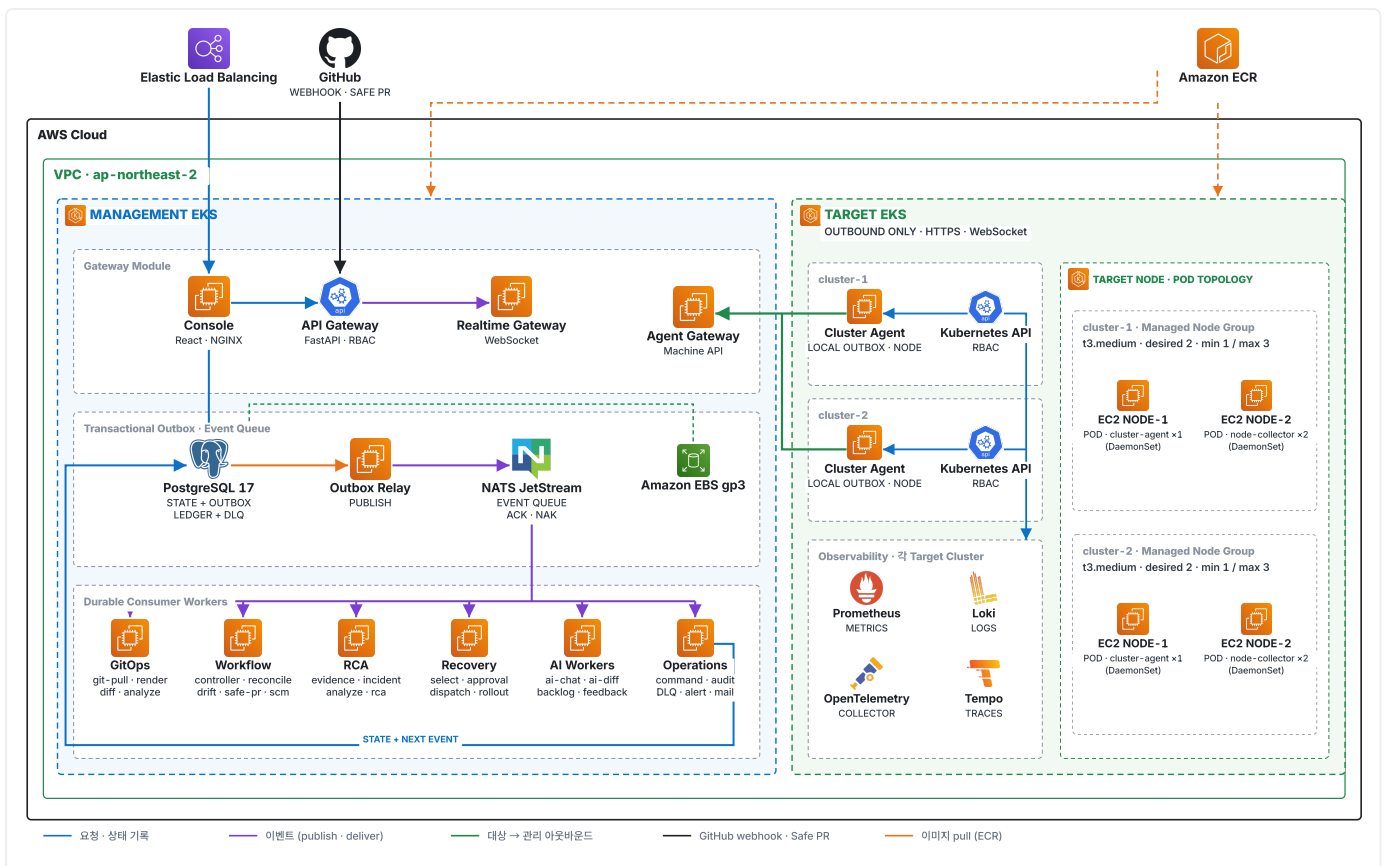


대시보드 메인 화면입니다. 클러스터 3개, 동기화 필요 1건, 장애 2건의 현황을 한 화면에 시각화합니다

항목	내용
기간	팀 2026-08 · 개인 정리 2026-08 ~ 09
역할	팀장 · 이벤트 런타임 · 계약 · CI · 배포
스택	Python · FastAPI · PostgreSQL · Helm
저장소	woonyong-choi/k8s-clue · 팀 원본 minmings111/Kyro-jungle-final
검증	pytest 294건 · 계약 테스트 86건 · 골든셋 116건
내 담당	아키텍처 · 이벤트 런타임 · 증거 정규화 · CI 게이트 · 배포
팀원	규칙 카탈로그 · 에이전트와 수집 · PR 생성 · 게이트웨이 인증
개인 정리	완결 경로 축소 · 안전 계약 9개 항목 계약 테스트 고정

문제 정의와 시스템 구조

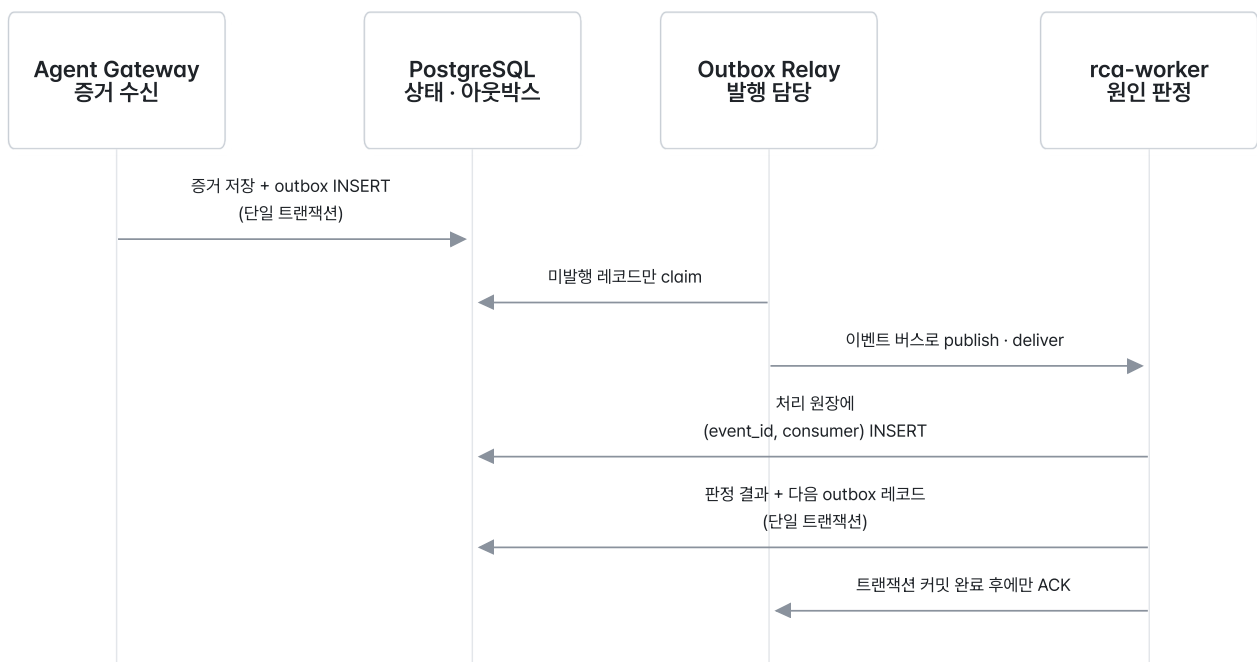
Kubernetes 장애 대응의 단서는 네 곳에 흩어져 있습니다. Pod 상태와 Event는 Kubernetes API에, 로그는 Loki에, 지표는 Prometheus에, 배포를 유발한 변경 이력은 Git에 있습니다. 운영자는 이 정보를 모아 원인을 추정하고, YAML을 수정해 재배포한 뒤 회복 여부를 다시 확인합니다. Clue는 수집부터 회복 확인까지를 단일 correlation_id 로 연결합니다. 자동화 권한은 수집, 판정, 변경 단계마다 다르게 제한했습니다. 관리 클러스터는 API 게이트웨이, PostgreSQL, 워커의 3개 계층으로 구성되며, 단일 컨트롤러 프로세스가 15개 서비스를 실행합니다. 워커를 약 40개까지 분리한 시기도 있었지만 이벤트 전달과 서비스 간 통신 비용이 늘어, 장애 격리와 독립 확장이 필요한 범위만 남기고 통합했습니다.



관리 클러스터와 대상 클러스터의 배포 구조입니다. 에이전트가 관리 클러스터 방향으로만 연결을 개시합니다

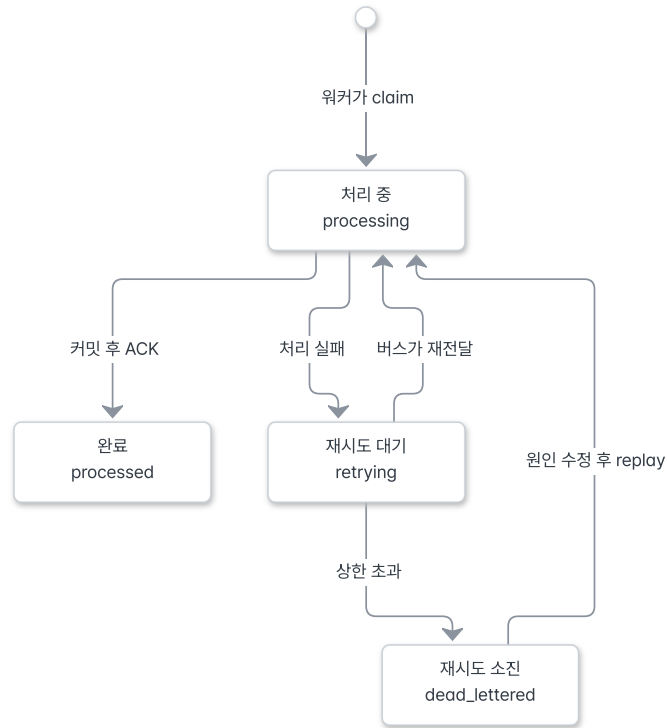
이벤트 런타임: Transactional Outbox와 처리 원장

서비스를 이벤트 기반 구조로 분리하면서 두 가지 새로운 실패 가능성이 발생했습니다. DB 커밋과 이벤트 발행 사이에서 이벤트가 유실될 수 있고, ACK가 유실되면 동일 작업이 중복 실행됩니다. 상태 변경과 아웃박스 레코드를 단일 트랜잭션에 기록하고 릴레이가 커밋된 레코드만 발행하도록 구현해 전반부의 유실을 방지했으며, 처리 원장 `event_processing`의 (`event_id`, `consumer`) 복합 PK를 활용해 후반부의 중복 실행을 차단했습니다. 워커가 `processing` 상태를 기록한 뒤 다운되는 상황도 발생합니다. 해당 상태가 90초 이상 지속되면 타임아웃된 `claim`으로 간주하여 다른 인스턴스가 재처리하도록 구성했습니다. 재시도가 소진되면 `event_dead_letters`로 이관하고, 원인을 수정한 뒤 `replay`로 동일 이벤트를 재투입합니다.



트랜잭션 커밋 후 메시지를 발행하고, 처리가 끝난 뒤 ACK를 응답합니다. 어느 지점에서 프로세스가 중단되더라도 결과는 재전달로 수렴합니다

워커의 처리 진행 상황은 처리 원장 한 곳에 상태로 기록됩니다. 네 가지 상태와 전이가 이벤트 하나의 결말을 모두 설명합니다.



처리 원장에 기록되는 consumer별 이벤트 상태와 전이를 나타냅니다

아웃박스에서 가장 빈번한 연산은 미발행 레코드 조회입니다. 따라서 해당 조건만 포함하는 부분 인덱스를 별도로 두었으며, 발행이 완료된 레코드가 누적되어도 이 인덱스는 증가하지 않습니다.

[src/packages/storage/schema.py](#)

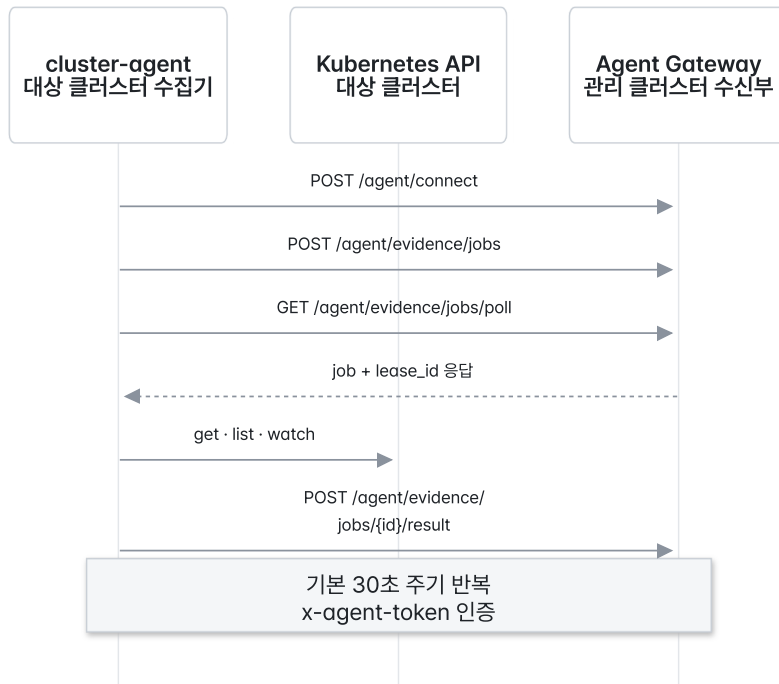
```
# src/packages/storage/schema.py
Index("ix_outbox_claim_all_sources",
      "sent_at", "leased_until", "id",
      postgresql_where=text("sent_at IS NULL"))
```

이벤트 발행을 기록하는 주체는 항상 DB 트랜잭션이며, 메시지 버스는 전달 역할만 담당합니다.

안전 원칙: 읽기 전용 수집과 Draft PR 전용 변경

증거 수집 주체가 클러스터 상태를 변경할 수 있다면 권한 분리 경계가 무너집니다. 따라서 에이전트에는 `get`, `list`, `watch` 권한만 부여했습니다. 권한 부여만으로는 충분하지 않아, Helm으로 렌더링한 manifest 전체를 검사해 읽기 전용이 아닌 verb, `pods/exec` 같은 직접 실행 리소스, wildcard가 하나라도 발견되면 CI가 중단되도록 구성했습니다. 검사 대상을 소스 코드가 아닌 렌더링 결과로 정한 이유는, 차트가 조건 분기로 생성하는 최종 manifest가 실제로 클러스터에 적용되기 때문입니다.

팀원이 구현한 에이전트는 관리 서버 방향으로 연결을 개시하며, DaemonSet에는 포트와 Service가 모두 없고 관리 서버가 하달할 수 있는 명령 채널도 존재하지 않습니다. 변경 경로에도 동일한 기준을 적용했습니다. 대상이 Deployment가 아니면 거부하고, 값 하나를 치환하는 형태만 허용하며, PR 생성 직전에 base SHA를 재조회해 증거 수집 시점과 불일치하면 생성을 중단합니다.



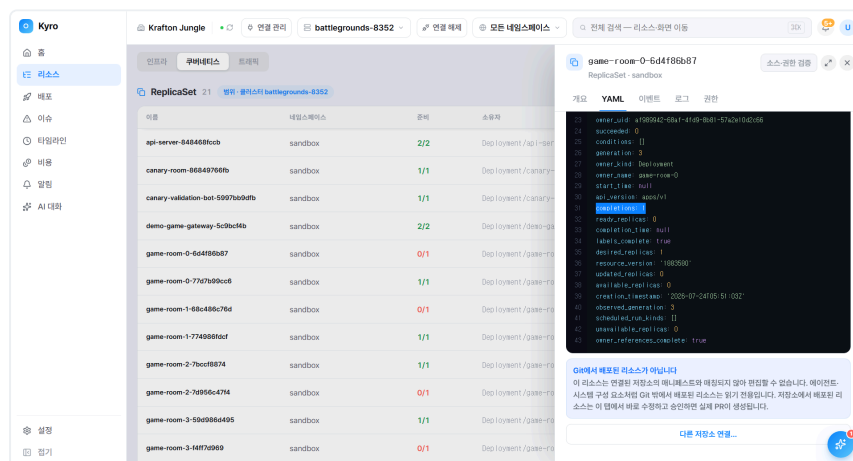
에이전트가 등록부터 결과 전송까지 직접 개시한 연결만을 사용해 처리합니다. 관리 클러스터에서 대상 클러스터로 진입하는 경로는 존재하지 않습니다

PR 생성 코드는 팀원이 작성했고, 저는 base SHA 재확인과 Draft 강제, merge API 미호출 세 가지가 유지되는지 검증하는 계약 테스트를 추가했습니다.

```
src/services/gitops/scm-worker/github_provider.py
```

```
# src/services/gitops/scm-worker/github_provider.py
if current_base_sha != expected_base_sha:
    raise RuntimeError(STALE_BASE_MESSAGE)
response = await client.post(f"/repos/{repo}/pulls", json={
    "head": branch, "base": base_branch, "draft": True})
payload = response.json()
if payload.get("draft") is not True:
    raise RuntimeError(DRAFT_REQUIRED_MESSAGE)
```

콘솔에서도 동일한 경계가 드러납니다. Git으로 배포되지 않은 리소스는 수정 탭 자체가 활성화되지 않습니다.



콘솔의 ReplicaSet 화면입니다. Git으로 배포된 리소스만 수정 탭이 열리며, 승인 시 실제 PR이 생성됩니다

자동 복구가 완전 자동화로 종결되지 않는 구조는 기술적 한계가 아닌 의도된 설계입니다.

검증 범위: 완결 경로 ImagePullBackOff

시나리오 문서는 실무 장애 35종을 정의했지만, 증거 수집부터 회복 확인까지 완결 검증한 경로는 ImagePullBackOff 하나입니다. 이 장애만 남은 이유는 증거가 Kubernetes API와 Event만으로 충족되고 수정이 image tag 하나로 종결되기 때문입니다. 나머지 시나리오는 지표와 로그의 임계치가 필요한데, 팀원이 작성한 카탈로그는 수치 임계치를 의도적으로 비워 두었습니다. 단일 값이 아니라 지속 시간과

반복 횟수를 함께 판단해야 한다는 원칙 때문이며, 해당 임계치를 확정하려면 실제 클러스터에 부하를 인가하는 검증이 필요했으나 비용과 시간 제약으로 시연 직전 1회만 수행할 수 있었습니다. 기능 범위는 축소되었지만 이 도구가 수행하지 않는 동작을 테스트로 진술할 수 있게 되었고, 안전 계약 9개 항목은 `make demo` 가 호출하는 계약 테스트로 고정되어 있습니다.

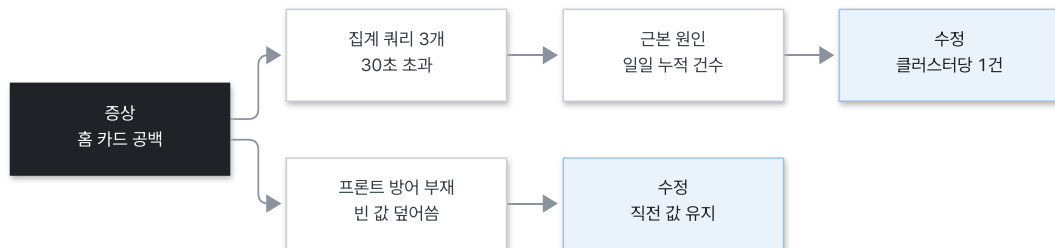
```
# Makefile
gate-backend: ## Backend와 manifest 전체 gate
    bash scripts/test.sh
    bash scripts/manifest-check.sh
demo: ## Kind ImagePullBackOff → Draft PR → 검증 데모
    bash scripts/oss-demo.sh
```

장애 대응: 홈 화면 집계 쿼리 타임아웃

단위 테스트와 kind 로컬 클러스터가 재현하지 못한 조건은 데이터 규모였습니다. 시연용 EKS에서 에이전트가 8~30초 주기로 적재한 스냅샷이 하루 만에 수십만 건에 도달했고, 시연 전날 홈 화면의 요약 카드가 모두 빈 값으로 출력되는 장애가 발생했습니다. 직접 원인은 세 갈래였습니다.

- **집계 쿼리 3개** — 리소스 레코드마다 최신 스냅샷을 재조회하거나 usage 전체에 시간 범위 없는 window 함수를 적용해 `statement_timeout` 30초 초과.
- **프론트 catch** — 예외를 흡수하고 빈 객체를 반환해 직전 정상 값을 덮어쓰는 동작. 주석과 구현의 불일치.
- **수집 완료 게이트** — 완전 수집 스냅샷에만 카운트를 채우는 구조로 인해 namespace 범위 에이전트에서 영구 null 반환.

근본 원인은 append-only 테이블에 기록 주기 8~30초를 곱해 일일 누적 레코드 수를 산정하지 않은 것입니다.



증상 하나가 서버 측 원인과 프론트엔드 측 원인으로 분기하며, 각각 다른 수정으로 종결됩니다

수정 방식은 이미 정상 동작하던 `/clusters` 경로와 동일합니다. 요청받은 클러스터 ID를 `VALUES` 로 전개하고, 클러스터마다 `LATERAL` 로 최신 스냅샷 한 건만 조회합니다. `ix_inventory_snapshots_live_scope_latest` 가 정렬 순서까지 포함한 부분 인덱스이므로 첫 레코드에서 탐색이 종료됩니다.

```
src/domains/inventory/repository.py

# src/domains/inventory/repository.py
latest = (
    select(table.c.snapshot_id, table.c.cluster_id)
    .where(table.c.workspace_id == workspace_id,
          table.c.cluster_id == requested.c.cluster_id,
          live_inventory_snapshot_clause(table))
    .order_by(table.c.created_at.desc(), table.c.snapshot_id.desc())
    .limit(1).lateral("latest_inventory_snapshot")
```

30초 주기로 스냅샷을 재기록할 필요가 있는지, 처리 원장을 언제 정리해야 하는지는 당시 답을 내지 못했습니다.

검증 결과와 수치

아래는 정리본 저장소에서 실행한 결과입니다. 전부 로컬 테스트와 계약 수준의 검증이며 실사용 성능 지표가 아닙니다. 골든셋 정확도가 상승한 요인은 규칙 추가가 아니라, 범위가 넓은 로그 패턴이 더 구체적인 패턴을 덮고 점수가 동일할 때 선언 순서가 결과를 좌우하던 문제를 수정한 것입니다. 87건 중 5건이 상호 혼동되고 있었습니다.

항목	값	조건
pytest	294건 passed	PostgreSQL을 기동한 정리본 전체 · 2026-09-22 재실행
계약 테스트	86건 = 30 + 29 + 27	증거·판정 / Draft PR / 배포 후 검증
골든셋	116건 = 장애 87 + 정상 29	카탈로그 역산 합성 입력 · 실제 장애 정확도 아님
골든셋 정확도	94.3% → 100% · 오탐 0%	동일 116건 · 패턴 우선순위와 동점 처리 수정 결과
홈 화면 쿼리	30초 초과 → 수 ms	시연용 클러스터 당시 데이터 · 단일 관측

시연에서는 이 런타임 위에서 클러스터 3개를 동시에 조회하고, 판정 결과를 근거와 함께 화면에 출력했습니다.

canary-room-86849

클러스터 battlegrounds-...

종류 ReplicaSet

장애 발생 · Ingress 502/503

codex-connect-e2e-latest-20260724-4759 · Deployment cluster-agent

이슈 상세 복구 플랜

상황 요약

sandbox 네임스페이스의 ReplicaSet canary-room-86849766fb에서 FailedScheduling 증상이 발생하였으며, 이는 노드의 CPU 자원이 부족하여 신규 파드가 스케줄되지 못한 것으로 확인되었습니다. 주요 근거로는 Kubernetes

멀티 클러스터 시연 화면입니다. 노드 3/4, 파드 68개, 장애 3건을 표시합니다

콘솔의 원인 분석 요약입니다. FailedScheduling과 확인된 근거 2건, 승인 권장을 제시합니다

범위와 한계

설계에서 의도적으로 제외한 항목과 끝내 측정하지 못한 항목을 구분해 기술합니다. 전자는 자동화의 권한을 차단한다는 원칙에서 도출한 결정이므로 재설계하더라도 동일하게 정할 것입니다. 후자는 시간과 환경의 제약이었고, 특히 실사용 트래픽이 없다는 점이 이 프로젝트의 가장 큰 공백입니다. 그래서 앞 절의 수치는 전부 로컬과 시연 환경에서 산출한 값이라고 명시해 두었습니다.

- **설계에서 제외** — 전체 장애의 자동 복구, LLM의 자유로운 YAML 수정, 클러스터 명령 직접 실행, 자동 merge와 rollback.
- **시간 제약으로 제외** — 비용 대시보드, 웹 터미널.
- **미측정** — 실사용 트래픽, 복구 시간, 처리량, 비용.
- **미완** — 외부 클러스터와 GitHub App 연동의 종단 검증.

비용은 수치로 측정하지 못했고 판단은 전부 정성적이었습니다. 방향은 하나였습니다. 서비스를 세분화할수록 비용이 증가한다는 것이며, 워커 통합과 원본 텔레메트리 미저장이 여기서 도출한 결정입니다. 에이전트가 명령 채널을 보유하지 않는다는 제약은 클래스 주석에 그대로 명시해 두었습니다.

```
src/services/target/cluster-agent/agent.py
```

```
# src/services/target/cluster-agent/agent.py
class TargetClusterAgent:
    """Run Kubernetes evidence jobs; it has no command or mutation channel."""
```